

Started on	Thursday, 12 June 2025, 12:47 AM
State	Finished
Completed on	Thursday, 12 June 2025, 12:51 AM
Time taken	4 mins 14 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Viết chương trình đọc một đồ thị **vô hướng** từ bàn phím và kiểm tra xem đồ thị có liên thông không.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v) .

Đầu ra (Output)

- Nếu đồ thị liên thông in ra **CONNECTED**, ngược lại in ra **DISCONNECTED**.

Gợi ý

- Duyệt đồ thị từ một đỉnh. Sau khi duyệt, kiểm tra xem có đỉnh nào chưa được không.

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

For example:

Input	Result
4 3 2 1 1 3 2 4	CONNECTED

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
```

```

31 //-----DFS-----
32
33 int cnt;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36
37     visited[s] = 1;
38     // printf("%d \n", s);
39     for (int u = 1; u <= pG->n; u++) {
40         if (pG->A[s][u] > 0 && !visited[u]) {
41             DFS(pG, u);
42         }
43     }
44 }
45
46 int main() {
47     Graph G;
48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     // DFS(&G, 1);
62
63     cnt = 0;
64     for (int i = 1; i <= G.n; i++) {
65         if (!visited[i]) {
66             cnt++;
67             DFS(&G, i);
68         }
69     }
70     if(cnt>1){
71         printf("DISCONNECTED");
72     }else{
73         printf("CONNECTED");
74     }
75 }
76

```

	Input	Expected	Got	
✓	4 3 2 1 1 3 2 4	CONNECTED	CONNECTED	✓
✓	4 2 1 2 3 4	DISCONNECTED	DISCONNECTED	✓
✓	4 2 1 4 2 3	DISCONNECTED	DISCONNECTED	✓

	Input	Expected	Got	
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 5 6	CONNECTED	CONNECTED	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35
36 void DFS(Graph *pG, int u) {
37     //1. Đánh dấu u đã duyệt
38     //printf("Duyet %d\n", u); //Làm gì đó trên u
39     mark[u] = 1; //Đánh dấu nó đã duyệt
40
41     //2. Xét các đỉnh kề của u
42     for (int v = 1; v <= pG->n; v++)
43         if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
44             DFS(pG, v); //gọi đệ quy duyệt nó
45
46 }
47
48 // Kiểm tra pG có liên thông không
49 int connected(Graph *pG) {
50     //1. Khởi tạo tất cả đỉnh đều chưa duyệt
51     for (int u = 1; u <= pG->n; u++)
52         mark[u] = 0;
53     DFS(pG, 1);
54     for (int u = 2; u <= pG->n; u++)
55         if (mark[u] == 0)
56             return 0;
57     return 1;
58 }
59
60 int main() {
61     int n, m;
62     scanf("%d %d", &n, &m);
63     Graph pG;
64     init_graph(&pG, n);
65     while (m--) {
66         int u, v;
67         scanf("%d %d", &u, &v);
68         add_edge(&pG, u, v);
69     }
70     if (connected(&pG))
71         printf("CONNECTED\n");
72     else
73         printf("NOT CONNECTED\n");
74     return 0;
75 }

```

```

52     mark[u] = 0;
53     //2. Duyệt đồ thị từ đỉnh bất kỳ, ví dụ: 1
54     DFS(pG, 1);
55     //3. Kiểm tra xem có đỉnh nào chưa duyệt không
56     for (int u = 1; u <= pG->n; u++)
57         if (mark[u] == 0) //Vẫn còn đỉnh chưa duyệt
58             return 0; //Đồ thị không liên thông, thoát luôn
59
60     return 1; //Tất cả các đỉnh đều đã duyệt => liên thông
61 }
62
63
64 int main() {
65     //1. Khai báo đồ thị G
66     Graph G;
67     //2. Đọc dữ liệu và dựng đồ thị
68     int n, m, u, v;
69     scanf("%d%d", &n, &m);
70     init_graph(&G, n);
71     for (int e = 0; e < m; e++) {
72         scanf("%d%d", &u, &v);
73         add_edge(&G, u, v);
74     }
75
76     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
77     for (int u = 1; u <= G.n; u++) {
78         mark[u] = 0;
79     }
80
81     //4. Gọi hàm connected để kiểm tra
82
83     if (connected(&G))
84         printf("CONNECTED\n");
85     else
86         printf("DISCONNECTED\n");
87
88     return 0;
89 }
90
91

```

Correct

Marks for this submission: 1.00/1.00.

◀ * Bài tập 6. Xây dựng cây duyệt đồ thị

Jump to...

* Bài tập 8. Bộ phận liên thông ▶